

AULA 1 - Introdução às Bases de Dados

1. O que é uma Base de Dados?

Uma base de dados é uma coleção organizada de dados relacionados.

Não é só “guardar dados”, é guardar de forma organizada para:

- procurar rapidamente
- atualizar sem erros
- permitir vários utilizadores
- manter tudo consistente

Exemplo simples

Imagina uma escola:

- alunos
- professores
- disciplinas
- notas

Se tudo isto estivesse em folhas soltas, seria um caos. A base de dados é o **arquivo inteligente** da escola.

2. Porque não usar ficheiros normais?

Antes das bases de dados, usavam-se ficheiros (.txt, .csv, Excel).

Problemas reais dos ficheiros

- Informação repetida
- Dados inconsistentes
- Difícil de procurar coisas complexas
- Várias pessoas ao mesmo tempo -> erros
- Falhas deixam dados corrompidos

Exemplo

- Um aluno muda de curso
- O nome aparece em 10 ficheiros
- Tens de alterar em 10 sítios -> erro garantido

3. O que resolve as Bases de Dados?

As bases de dados resolvem tudo isto porque:

- Guardam dados uma única vez
- Impõem regras (ex.: idade não pode ser negativa)
- Controlam acessos simultâneos
- Recuperam dados após falhas
- Protegem informação sensível

4. O que é um SGBD?

SGBD = Sistema de Gestão de Bases de Dados. É o software que gere tudo.

O que ele faz:

- Guarda dados no disco
- Garante segurança
- Controla concorrência
- Executa consultas
- Aplica regras de integridade

Tu não falas com os ficheiros, falas com o SGBD.

Exemplos

- MySQL
- PostgreSQL
- Oracle
- SQL Server

5. Analogia importante

Biblioteca:

- Livros → dados
- Catálogo → índices
- Bibliotecário → SGBD
- Leitor → utilizador

O leitor não entra no armazém, pede ao bibliotecário.

6. Três níveis de abstração

O SGBD separa o sistema em **3 níveis**:

Nível Físico

- Como os dados são guardados no disco
- Blocos, ficheiros, bytes

-O utilizador não vê

Nível Lógico

- Tabelas
- colunas
- relações

-É onde o modelo relacional vive

Nível de Vista

- Diferentes “visões” dos dados
- Um utilizador vê só parte da informação

-Segurança e personalização

7. Porque isto é importante?

Porque permite:

- mudar o disco sem mudar aplicações
- mudar tabelas internas sem afetar utilizadores
- segurança
- escalabilidade

Este conceito chama-se **independência dos dados**.

AULA 2 - Modelo Relacional: tabelas, linhas e colunas

1. O que é o Modelo Relacional?

O **modelo relacional** é a forma mais comum de organizar dados em bases de dados.

- Ideia central: Todos os dados são guardados em tabelas.

Nada de estruturas complicadas, nada de ficheiros escondidos:

- apenas tabelas
- relacionadas entre si

2. O que é uma Tabela?

Uma tabela representa um tipo de objeto do mundo real.

Exemplos

- Aluno
- Professor
- Curso
- Disciplina
- Cliente

Cada tabela guarda informação sobre uma coisa específica.

3. Linhas (Tuplos)

Cada linha da tabela representa um elemento concreto.

Exemplo — Tabela Aluno

ID	Nome	Curso
1	Ana	EI
2	João	MEI

- Linha 1 → a aluna Ana
- Linha 2 → o aluno João

Cada linha chama-se tuplo.

4. Colunas (Atributos)

Cada coluna descreve uma característica do objeto.

No exemplo:

- ID → identificador
- Nome → nome do aluno
- Curso → curso frequentado

- Cada coluna chama-se atributo.

5. Domínio dos atributos

Cada atributo tem um domínio = conjunto de valores permitidos

Exemplos

- idade → números inteiros ≥ 0
- nome → texto
- nota → 0 a 20

Isto evita valores absurdos (ex.: idade = “banana”).

6. Valores NULL

Às vezes não sabemos um valor. Para isso existe o valor especial **NULL**.

O que significa NULL?

- valor desconhecido
- valor ainda não atribuído

Exemplo

- aluno ainda não tem nota
- funcionário ainda não tem salário

-NULL não é zero, nem string vazia

7. Esquema vs Instância

-Esquema:

É a estrutura da tabela. Define:

- nomes das colunas
- tipos de dados

Exemplo: `Aluno(ID, Nome, Curso)`

-Instância:

São os dados reais num dado momento.

ID	Nome	Curso
1	Ana	EI
2	João	MEI

-Se amanhã entra outro aluno, o esquema é o mesmo, mas a instância muda.

8. As tabelas não têm ordem

As linhas de uma tabela NÃO têm ordem. Mesmo que apareçam listadas:

- a base de dados não garante essa ordem
- se quiseres ordem → tens de pedir explicitamente (mais tarde em SQL)

9. Porque o modelo relacional é tão usado?

Porque:

- é simples
- é matematicamente bem definido
- evita redundância
- facilita consultas
- funciona bem com SQL

AULA 3 - Chaves e Integridade dos Dados

1. Porque precisamos de chaves?

Imagina uma tabela de alunos sem identificador único:

Nome	Curso
Ana	EI
Ana	MEI

-Qual é qual?

-Como distingues as duas “Ana”?

2. O que é uma chave?

Chave = conjunto de atributos que identifica unicamente uma linha (tuplo)

Ou seja:

- não se repete
- aponta para uma e só uma linha

3. Superchave

Uma **superchave** é qualquer conjunto de atributos que identifica unicamente uma linha.

Exemplo

Tabela Aluno:

ID	Nome	Email
----	------	-------

Superchaves possíveis:

- {ID}
- {Email}
- {ID, Nome}
- {ID, Email}

- Todas identificam unicamente o aluno

- Mas nem todas são boas escolhas

4. Chave Candidata

Chave candidata = superchave mínima

mínima = não tem atributos a mais

No exemplo anterior

- {ID} ✓
- {Email} ✓
- {ID, Nome} ✗ (Nome é extra)

-As chaves candidatas são as melhores opções possíveis.

5. Chave Primária (muito importante ⚠️)

Chave primária é: a chave candidata escolhida para identificar oficialmente as linhas da tabela

Regras da chave primária

- Não se repete
- Nunca é NULL
- Existe apenas uma por tabela

Exemplo

```
Aluno(  
    ID,           ← chave primária  
    Nome,  
    Curso  
)
```

-O ID identifica cada aluno sem ambiguidades.

6. Porque a chave primária é essencial?

Sem chave primária:

- não sabes atualizar corretamente
- não consegues ligar tabelas
- erros aparecem facilmente

7. Chave Estrangeira

Chave estrangeira é um atributo que:

- aponta para a chave primária de outra tabela
- cria uma ligação entre tabelas

Exemplo clássico

Tabela Curso

Código	Nome
EI	Eng. Informática
MEI	Mestrado EI

Tabela Aluno

ID	Nome	Curso
1	Ana	EI
2	João	MEI

-**Aluno.Curso** é uma **chave estrangeira**

-Aponta para **Curso.Codigo**

8. Para que servem as chaves estrangeiras?

- Evitam dados repetidos
- Garantem que só existem valores válidos
- Mantêm os dados consistentes

Exemplo de erro evitado

Sem chave estrangeira:

- aluno com curso = "Astrologia Quântica" 😅

Com chave estrangeira:

- só cursos existentes são aceites

9. Integridade Referencial

Integridade referencial garante que toda chave estrangeira aponta para uma linha válida noutra tabela

Regras importantes:

- Não podes inserir um aluno com um curso que não exista
- Não podes apagar um curso se ainda houver alunos nesse curso (a não ser que definas regras especiais)

- O SGBD garante isto automaticamente.

10. Analogia do dia a dia

Lista telefónica

- Número → chave primária
- Nome → atributo normal

AULA 4 - Introdução ao SQL

1. O que é SQL?

SQL (Structured Query Language) é a linguagem usada para:

- falar com a base de dados
- criar estruturas
- inserir dados
- procurar dados
- alterar dados

-SQL é a **ponte entre o utilizador e o SGBD**.

2. SQL não é uma linguagem “normal”

Isto é MUITO importante: SQL é declarativo, não é procedural

O que isto significa?

- Não dizes como o sistema deve procurar
- Dizes apenas o que queres

Exemplo:

```
SELECT nome
FROM aluno
WHERE curso = 'EI';
```

Tu não dizes:

- por onde procurar
- em que ordem
- com que algoritmo

- O SGBD decide isso por ti.

3. As duas grandes partes do SQL

- DDL — Data Definition Language

Serve para definir a estrutura da base de dados.

Exemplos:

- criar tabelas
- alterar tabelas
- apagar tabelas

Comandos típicos:

- CREATE
- ALTER
- DROP

- DML — Data Manipulation Language

Serve para trabalhar com os dados.

Exemplos:

- inserir dados
- consultar dados
- atualizar dados
- apagar dados

Comandos típicos:

- `SELECT`
- `INSERT`
- `UPDATE`
- `DELETE`

4. Exemplo prático (visão geral)

Imagina que queremos guardar alunos.

- DDL — criar a tabela

```
CREATE TABLE aluno (  
    id INT,  
    nome VARCHAR(50),  
    curso VARCHAR(10)  
);
```

Aqui está a definir a estrutura, não os dados.

- DML — consultar dados

```
SELECT nome  
FROM aluno  
WHERE curso = 'EI';
```

Aqui está a pedir informação.

5. Como funciona uma consulta SQL (intuição)

Uma consulta SQL segue esta lógica mental:

1. **FROM** → onde estão os dados
2. **WHERE** → que linhas interessam
3. **SELECT** → o que mostrar

6. Lendo uma consulta em português

Consulta:

```
SELECT nome  
FROM aluno  
WHERE curso = 'EI';
```

Lê assim: “Mostra o nome dos alunos que pertencem ao curso EI.”

7. SQL devolve sempre uma tabela

Mesmo que:

- seja só um valor
- seja uma coluna
- seja uma linha

O resultado de uma consulta SQL é **sempre uma tabela**. Isto liga diretamente ao **modelo relacional**.

8. Onde o SQL é usado?

- Aplicações web
- Bancos
- Sistemas escolares
- Hospitais
- Jogos
- Redes sociais

Sempre que há dados estruturados, há SQL (ou algo inspirado nele).

AULA 5 - SQL básico: SELECT, FROM e WHERE

1. Estrutura básica de uma consulta SQL

A forma mais simples de uma consulta é:

```
SELECT colunas
FROM tabelas
WHERE condição;
```

Pensa nisto como uma **frase**: “Mostra **X** de **Y** onde **Z**.”

2. FROM — De onde vêm os dados

- **FROM** diz em que tabelas estão os dados.

Exemplo

```
FROM aluno
```

- “Os dados vêm da tabela aluno.” Sem **FROM**, o SQL não sabe onde procurar.

3. SELECT — O que queres ver

- **SELECT** diz que colunas aparecem no resultado.

Exemplo

```
SELECT nome
FROM aluno;
```

Resultado:

nome
Ana
João

```
SELECT *
SELECT *
FROM aluno;
```

- ***** significa **todas as colunas**.

4. WHERE — Filtrar linhas

- **WHERE** escolhe quais linhas interessam.

Exemplo

```
SELECT nome
FROM aluno
WHERE curso = 'EI';
```

- Só alunos do curso EI.

5. Operadores mais usados no WHERE

Comparação

- = igual
- <> diferente
- <, <=, >, >=

Lógicos

- AND → e
- OR → ou
- NOT → não

Exemplos

```
SELECT nome
FROM aluno
WHERE curso = 'EI' AND id > 10;
```

-Alunos de EI com id maior que 10

```
SELECT nome
FROM aluno
WHERE curso = 'EI' OR curso = 'MEI';
```

-Alunos de EI ou MEI

6. Strings e aspas

Texto (strings) usa aspas simples: `WHERE nome = 'Ana'`

Números não usam aspas: `WHERE id = 5`

7. DISTINCT — remover duplicados

Por defeito, SQL permite duplicados.

```
SELECT curso
FROM aluno;
```

Resultado:

```
EI
EI
MEI
```

Com **DISTINCT**

```
SELECT DISTINCT curso
FROM aluno;
```

Resultado:

```
EI
MEI
```

8. Ordem lógica vs ordem de escrita

Mesmo que escreva:

```
SELECT nome
FROM aluno
WHERE curso = 'EI';
```

O sistema pensa assim:

1. FROM
2. WHERE
3. SELECT

AULA 6 - Tipos de Domínios (Tipos de Dados) em SQL

1. O que é um Domínio?

Domínio = conjunto de valores permitidos para um atributo

No modelo relacional:

- cada coluna tem um domínio
- SQL implementa domínios através de tipos de dados

- O domínio evita valores inválidos.

2. Porque os domínios são importantes?

Sem domínios:

- idade = "banana"
- salário = -5000

Com domínios:

- dados válidos
- menos erros
- mais integridade

3. Principais tipos de domínios em SQL

-Tipos Numéricos

Tipo	Para que serve
INT / INTEGER	números inteiros
SMALLINT	inteiros pequenos
BIGINT	inteiros grandes
DECIMAL(p, s) / NUMERIC(p, s)	números exatos
FLOAT, REAL	números aproximados

Exemplo:

salario NUMERIC(8,2): até 8 dígitos, 2 depois da vírgula

-Tipos de Texto (Strings)

Tipo	Descrição
CHAR(n)	tamanho fixo
VARCHAR(n)	tamanho variável
TEXT	texto grande

Exemplo:

nome VARCHAR(50): VARCHAR é o mais usado.

-Tipos de Data e Tempo

Tipo	Exemplo
DATE	2025-05-10
TIME	14:30:00
TIMESTAMP	2025-05-10 14:30:00

- Muito usados em sistemas reais.

-Tipos Booleanos

Tipo	Valores
BOOLEAN	TRUE / FALSE

4. Domínios personalizados (teoria importante)

Em SQL padrão, podes **definir domínios**:

```
CREATE DOMAIN nota_20 AS INT
CHECK (VALUE BETWEEN 0 AND 20);
```

- Isto cria um domínio reutilizável.
- Nem todos os SGBDs usam muito isso.

5. Domínios vs Restrições

Domínio	Restrição
Tipo de valor	Regra extra
Ex.: INT	Ex.: NOT NULL
Define formato	Define validade

- Eles **trabalham juntos**.

6. Exemplo completo

```
CREATE TABLE aluno (
    id INT,
    nome VARCHAR(50),
    idade INT,
    nota NUMERIC(4,2),
    data_nasc DATE
);
```

Cada coluna tem:

- um tipo
- um domínio implícito

AULA 7 — ORDER BY, BETWEEN, LIKE e operadores especiais

1. ORDER BY — ordenar resultados

Por defeito, **SQL não garante ordem nenhuma**. Se queres ordem, **tens de pedir explicitamente**.

Sintaxe

```
ORDER BY coluna;
```

Exemplo

```
SELECT nome
FROM aluno
ORDER BY nome;
```

- Ordena alfabeticamente (A → Z)

Ordem descendente

```
ORDER BY nome DESC; -> Z → A
```

Ordem ascendente

```
ORDER BY nome ASC;
```

Ordenar por várias colunas

```
ORDER BY curso, nome; -> Primeiro ordena por curso, depois por nome.
```

2. BETWEEN — valores dentro de um intervalo

Usado para números, datas, notas, salários, etc.

Sintaxe

```
WHERE coluna BETWEEN valor1 AND valor2;
```

Exemplo

```
SELECT nome
FROM aluno
WHERE id BETWEEN 10 AND 20;
```

-IDs entre 10 e 20 (inclusive)

Importante

```
BETWEEN 10 AND 20
```

é o mesmo que:

```
>= 10 AND <= 20
```

3. LIKE — procurar padrões em texto

Usado com **strings**.

Símbolos especiais

- `%` → qualquer sequência de caracteres
- `_` → exatamente um caractere

Exemplos

-Começa por “Ana”

```
WHERE nome LIKE 'Ana%';
```

-Contém “silva”

```
WHERE nome LIKE '%silva%';
```

-Termina em “s”

```
WHERE nome LIKE '%s';
```

-_ (um caractere)

```
WHERE codigo LIKE 'A_1';    -A01, A11, A21
```

4. IS NULL e IS NOT NULL

NULL não se testa com =

- ERRADO

```
WHERE nota = NULL;
```

- CORRETO

```
WHERE nota IS NULL;
```

ou

```
WHERE nota IS NOT NULL;
```

5. Exemplo completo (juntando tudo)

```
SELECT nome, nota
FROM aluno
WHERE curso = 'EI'
      AND nota BETWEEN 10 AND 20
ORDER BY nota DESC;
```

“Mostra o nome e a nota dos alunos de EI com nota entre 10 e 20, ordenados da maior para a menor.”

AULA 8 — JOIN: como juntar tabelas

1. Porque é que precisamos de JOIN?

Numa base de dados bem desenhada:

- não repetimos dados
- usamos chaves estrangeiras

Resultado: a informação fica dividida por várias tabelas.

Exemplo:

Aluno

id	nome	curso
1	Ana	EI
2	João	MEI

Curso

código	nome
EI	Eng. Informática
MEI	Mestrado EI

Pergunta: “Qual o nome dos alunos e o nome completo do curso?”

- Está em duas tabelas, precisamos de as juntar

2. O que é um JOIN?

JOIN é uma operação que combina linhas de duas (ou mais) tabelas relacionadas

A relação é feita normalmente através de:

- chave primária
- chave estrangeira

3. Produto cartesiano (base de tudo)

Antes do **JOIN**, é importante entender isto.

Produto cartesiano combina:

- cada linha da tabela A
- com cada linha da tabela B

Exemplo (sem condição)

Aluno (2 linhas)

Curso (2 linhas)

- Resultado: **4 linhas**.

4. INNER JOIN — o JOIN mais importante

INNER JOIN devolve apenas as linhas que têm correspondência nas duas tabelas

Sintaxe moderna

```
SELECT colunas
FROM tabela1
INNER JOIN tabela2
ON condição;
```

Exemplo prático

```
SELECT aluno.nome, curso.nome
FROM aluno
INNER JOIN curso
ON aluno.curso = curso.codigo;
```

- Lê-se: “Mostra o nome do aluno e o nome do curso, juntando aluno e curso onde o código do curso coincide.”

5. O papel do ON

ON define como as tabelas se ligam.

Normalmente:

- chave estrangeira = chave primária

`aluno.curso = curso.codigo`

Se esta condição estiver errada:

- resultados errados
- ou demasiadas linhas

6. O que acontece por baixo do capô

O SGBD faz algo assim:

1. Junta todas as combinações possíveis (produto cartesiano)
2. Filtra só as que cumprem a condição do ON

7. JOIN vs WHERE (erro clássico)

Forma moderna (usar sempre)

```
SELECT aluno.nome, curso.nome
FROM aluno
JOIN curso
ON aluno.curso = curso.codigo;
```

8. Nomes de colunas ambíguos

Se duas tabelas têm colunas com o mesmo nome:

Correto

```
SELECT aluno.nome, curso.nome
FROM aluno JOIN curso
ON aluno.curso = curso.codigo;
```

9. Exemplo completo

```
SELECT aluno.nome, curso.nome
FROM aluno
JOIN curso
ON aluno.curso = curso.codigo
ORDER BY aluno.nome;
```

Lista alunos com o nome do curso, ordenados alfabeticamente.

AULA 9 — Restrições em SQL (Constraints)

1. O que são Restrições em SQL?

Restrições são regras impostas aos dados numa tabela. Dizem ao SGBD:

- que valores são permitidos
- que valores são proibidos
- como as tabelas se relacionam

O próprio SGBD impede erros automaticamente.

2. Porque as restrições são essenciais?

Sem restrições:

- dados inválidos
- erros silenciosos
- inconsistências graves

Com restrições:

- dados corretos
- menos bugs
- base de dados confiável

3. NOT NULL

Impede valores NULL numa coluna.

Exemplo

```
nome VARCHAR(50) NOT NULL
```

O valor **tem de existir**.

Usado quando:

- o atributo é obrigatório
- faz parte de uma identificação

4. PRIMARY KEY

Define a **chave primária** da tabela.

Propriedades

- Única
- Não NULL
- Uma por tabela

Exemplo

```
id INT PRIMARY KEY
```

Ou:

```
PRIMARY KEY (id)
```

5. UNIQUE

Garante que os valores não se repetem, mas permite NULL.

Exemplo

```
email VARCHAR(100) UNIQUE
```

Dois utilizadores não podem ter o mesmo email.

Diferença importante:

- **PRIMARY KEY** → não aceita NULL
- **UNIQUE** → pode aceitar NULL (dependendo do SGBD)

6. FOREIGN KEY

Cria uma ligação entre tabelas.

Exemplo

```
curso VARCHAR(10),  
FOREIGN KEY (curso) REFERENCES curso(codigo)
```

Garante integridade referencial:

- só valores existentes são aceites
- evita referências inválidas

Regras típicas

- não podes inserir valores inexistentes
- não podes apagar dados “referenciados” (por defeito)

7. CHECK

Impõe **condições lógicas** aos valores.

Exemplo

```
nota INT CHECK (nota BETWEEN 0 AND 20)
```

Outros: `idade INT CHECK (idade >= 0)`

8. Exemplo completo de tabela

```
CREATE TABLE aluno (  
  id INT PRIMARY KEY,  
  nome VARCHAR(50) NOT NULL,  
  email VARCHAR(100) UNIQUE,  
  idade INT CHECK (idade >= 0),  
  curso VARCHAR(10),  
  FOREIGN KEY (curso) REFERENCES curso(codigo)  
);
```

Aqui temos:

- domínios
- chaves
- integridade
- regras de negócio

9. Onde definir restrições?

As restrições podem ser:

- ao nível da coluna
- ao nível da tabela

Ambas são corretas - depende da clareza e do caso.

AULA 10 — VIEWS (Vistas)

1. O que é uma VIEW?

Uma VIEW é uma tabela virtual baseada numa consulta SQL.

Importante:

- não guarda dados
- guarda apenas a consulta
- os dados vêm sempre das tabelas originais

Pensa numa view como: “uma janela para os dados”

2. Porque precisamos de VIEWS?

As views existem para resolver **problemas reais**:

- **Simplificar consultas:** Consultas longas e complicadas ficam reutilizáveis.
- **Segurança:** Mostrar apenas algumas colunas ou linhas.
- **Abstração:** O utilizador não precisa de saber como os dados estão organizados.

3. Exemplo simples

Tabela **aluno**

id	nome	curso	nota
----	------	-------	------

Queremos mostrar **apenas nome e curso**.

Criar uma view:

```
CREATE VIEW alunos_publicos AS  
SELECT nome, curso  
FROM aluno;
```

- **alunos_publicos** comporta-se como uma tabela.

Usar a view

```
SELECT *  
FROM alunos_publicos;
```

- Funciona como uma tabela. Mas não guarda dados próprios

4. View ≠ Tabela (diferença essencial)

Tabela	View
Guarda dados	Não guarda dados
Ocupa espaço	Não ocupa (quase)
Dados próprios	Dados vêm de outras tabelas
Estrutura física	Estrutura lógica

5. Views para segurança

Imagina uma tabela **funcionario**:

id	nome	salario
----	------	---------

-Não queres que todos vejam o salário.

Criar view sem salário

```
CREATE VIEW funcionarios_publicos AS
SELECT id, nome
FROM funcionario;
```

-Os utilizadores acedem à view, não à tabela real.

6. Views com filtros (linhas)

Views também podem esconder linhas.

Exemplo

```
CREATE VIEW alunos_EI AS
SELECT *
FROM aluno
WHERE curso = 'EI';
```

- Só mostra alunos de EI.

7. Views com JOIN

Views podem juntar tabelas.

```
CREATE VIEW info_aluno AS
SELECT aluno.nome, curso.nome AS curso
FROM aluno
JOIN curso
ON aluno.curso = curso.codigo;
```

- Simplifica consultas futuras:

```
SELECT * FROM info_aluno;
```

8. Atualizações em VIEWS

Posso fazer **INSERT**, **UPDATE**, **DELETE** numa view?

Regra geral:

- **Views simples** → às vezes sim
- **Views com JOIN, agregações, DISTINCT** → normalmente não

- Isto depende do SGBD. Nem todas as views são atualizáveis.

9. Eliminar uma VIEW

```
DROP VIEW alunos_publicos;
```

- Apaga só a view

- As tabelas originais não são afetadas

AULA 12 — Processamento e Otimização de Interrogações

1. O que acontece quando escreves uma query SQL?

Quando escreves:

```
SELECT nome
FROM aluno
WHERE curso = 'EI';
```

O SGBD **não executa logo**. Ele passa por várias **fases internas**.

2. Fases do Processamento de Interrogações

De forma simplificada, o processo é:

1. **Análise e parsing**
2. **Tradução para álgebra relacional**
3. **Otimização**
4. **Execução**.

3. Análise e Parsing

O SGBD verifica se a query é válida.

Verifica:

- sintaxe correta?
- tabelas existem?
- colunas existem?
- tipos compatíveis?

-Se houver erro → a query é rejeitada aqui.

-Esta fase **não acede aos dados**.

4. Tradução para Álgebra Relacional

Depois de válida, a query SQL é transformada numa **expressão de álgebra relacional**.

Exemplo:

SQL:

```
SELECT nome
FROM aluno
WHERE curso = 'EI';
```

Álgebra:

```
 $\pi \text{ nome } (\sigma \text{ curso='EI' } (\text{Aluno}))$ 
```

5. O que é um Plano de Execução?

Plano de execução = estratégia concreta para executar a consulta.

Define:

- que tabelas ler primeiro
- que índices usar
- que algoritmo de JOIN usar
- ordem das operações

6. Porque precisamos de otimização?

Existem **muitas formas equivalentes** de executar a mesma consulta.

Exemplo lógico (equivalente):

```
 $\sigma \text{ condição } (R \times S)$ 
```

vs

```
 $(\sigma \text{ condição } (R)) \times S$ 
```

Mas:

- uma pode ser **muito mais rápida**
- outra pode ser **muito mais lenta**

- O SGBD tenta escolher a **mais barata**.

7. O que é Otimização de Interrogações?

Otimização é o processo de escolher o plano de execução com menor custo.

Custo mede:

- acessos a disco (principal fator)
- uso de CPU
- uso de memória

- O objetivo é o mais rápido.

8. Tipos de Otimização

- Otimização baseada em regras

- regras algébricas fixas
- independentes dos dados

Exemplo:

- fazer seleções o mais cedo possível
- reduzir o tamanho das tabelas antes de JOINS

- Otimização baseada em custos

- usa estatísticas
- estima número de linhas
- estima custo de cada plano

É a mais usada nos SGBDs modernos.

9. Regras clássicas de otimização (muito importantes)

- **Empurrar seleções para baixo:** Fazer WHERE o mais cedo possível.

- **Empurrar projeções para baixo:** Eliminar colunas desnecessárias cedo.

- **Reordenar JOINS:** Juntar primeiro as tabelas mais pequenas.

10. Exemplo intuitivo

Consulta:

```
SELECT nome
FROM aluno
JOIN inscricao ON aluno.id = inscricao.id_aluno
WHERE aluno.curso = 'EI';
```

Melhor estratégia:

1. filtrar alunos de EI
2. só depois fazer o JOIN

11. Estatísticas e o papel do catálogo

O SGBD usa:

- número de linhas por tabela
- distribuição dos valores
- existência de índices

Tudo isto está no **catálogo (dicionário de dados)**.

12. EXPLAIN (ligação prática)

Comando:

```
EXPLAIN SELECT ...
```

Mostra:

- plano escolhido
- ordem das operações
- custos estimados

AULA 13 — Funções SQL

1. O que são Funções em SQL?

Funções SQL são operações que:

- recebem valores (argumentos)
- devolvem um valor como resultado

São usadas **dentro de consultas** (**SELECT**, **WHERE**, **HAVING**, etc.)

Exemplo simples:

```
SELECT COUNT(*) FROM aluno;
```

2. Tipos de Funções em SQL

Existem dois grandes grupos:

- **Funções de agregação:** Trabalham sobre conjuntos de linhas
- **Funções escalares:** Trabalham sobre valores individuais

3. Funções de Agregação (muito importantes ⚠)

Usadas para **resumir dados**.

Principais funções

Função	O que faz
COUNT	conta linhas
SUM	soma valores
AVG	média
MIN	mínimo
MAX	máximo

Exemplos

- Contar alunos

```
SELECT COUNT(*) FROM aluno;
```

- Média das notas

```
SELECT AVG(nota) FROM aluno;
```

- Nota máxima

```
SELECT MAX(nota) FROM aluno;
```

4. GROUP BY — agrupar dados

Funções de agregação quase sempre vêm com **GROUP BY** (divide as linhas em grupos).

Exemplo

Média de nota por curso

```
SELECT curso, AVG(nota)
FROM aluno
GROUP BY curso;
```

-Um resultado por curso.

5. Regra fundamental

Tudo o que aparece no **SELECT** e não é função de agregação tem de estar no **GROUP BY**.

- ERRADO

```
SELECT nome, AVG(nota)
FROM aluno;
```

- CORRETO

```
SELECT curso, AVG(nota)
FROM aluno
GROUP BY curso;
```

6. HAVING — filtrar grupos

- **WHERE** filtra linhas

- **HAVING** filtra grupos

Exemplo

Cursos com média ≥ 10

```
SELECT curso, AVG(nota)
FROM aluno
GROUP BY curso
HAVING AVG(nota) >= 10;
```

- **HAVING** vem depois do **GROUP BY**.

7. Funções escalares (valor a valor)

- Funções de texto

Função	Exemplo
UPPER	UPPER(nome)
LOWER	LOWER(nome)
LENGTH	LENGTH(nome)
SUBSTRING	SUBSTRING(nome,1,3)

```
SELECT UPPER(nome)
FROM aluno;
```

- Funções numéricas

Função	Exemplo
ABS	ABS(valor)
ROUND	ROUND(nota,1)
CEILING	CEILING(valor)
FLOOR	FLOOR(valor)

- Funções de data

Função	Exemplo
CURRENT_DATE	data atual
CURRENT_TIMESTAMP	data + hora
EXTRACT	EXTRACT(YEAR FROM data_nasc)

8. Funções em WHERE (atenção ⚠)

Pode-se usar funções no **WHERE**, mas pode impedir o uso de índices (mais lento).

Exemplo: **WHERE YEAR(data_nasc) = 2000;**

Melhor (quando possível): **WHERE data_nasc BETWEEN '2000-01-01' AND '2000-12-31';**

9. Funções definidas pelo utilizador (UDFs)

Em alguns SGBDs podes criar funções próprias.

Exemplo (simplificado):

```
CREATE FUNCTION dobro(x INT)
RETURNS INT
RETURN x * 2;
```

Uso:

```
SELECT dobro(nota) FROM aluno;
```

10. Funções vs Procedimentos (prévia)

Funções	Procedimentos
Devolvem valor	Não precisam devolver
Usadas em SELECT	Executados com CALL
Sem efeitos colaterais	Podem alterar dados

AULA 14 - Procedimentos Armazenados (Stored Procedures)

1. O que é um Procedimento Armazenado?

Um procedimento armazenado é um bloco de código SQL guardado na base de dados que pode ser executado quando quisermos.

Ao contrário de uma função:

- não precisa devolver um valor
- pode executar várias instruções
- pode modificar dados

2. Para que servem procedimentos?

Procedimentos são usados para:

- Automatizar tarefas
- Reutilizar lógica
- Executar várias instruções de uma vez
- Garantir regras de negócio
- Melhorar segurança (menos SQL na aplicação)

3. Diferença entre Funções e Procedimentos

Funções	Procedimentos
Devolvem valor	Podem não devolver
Usadas em SELECT	Executadas com CALL
Sem efeitos colaterais	Podem alterar dados
Expressões	Ações

- Função = cálculo
- Procedimento = ação

4. Estrutura básica de um procedimento

Exemplo:

```
CREATE PROCEDURE listar_alunos()
BEGIN
    SELECT * FROM aluno;
END;
```

-Isto cria um procedimento guardado na BD.

5. Executar um procedimento

```
CALL listar_alunos();
```

O SGBD executa o código armazenado.

6. Procedimentos com parâmetros

Procedimentos podem receber dados.

Exemplo

```
CREATE PROCEDURE alunos_do_curso(c VARCHAR(10))
BEGIN
    SELECT nome
    FROM aluno
    WHERE curso = c;
END;
```

Execução:

```
CALL alunos_do_curso('EI');
```

7. Tipos de parâmetros (teoria)

Normalmente existem:

- **IN** → entrada (mais comum)
- **OUT** → saída
- **INOUT** → entrada e saída

8. Procedimentos que modificam dados

Exemplo:

```
CREATE PROCEDURE aumentar_notas()
BEGIN
    UPDATE aluno
    SET nota = nota + 1
    WHERE nota < 20;
END;
```

9. Porque usar procedimentos em vez de código na aplicação?

Vantagens

- Menos tráfego entre app e BD
- Lógica centralizada
- Mais segurança
- Melhor manutenção

Desvantagens

- Dependência do SGBD
- Código menos portátil
- Debug mais difícil

10. Procedimentos e Transações (ligação futura)

Procedimentos podem:

- iniciar transações
- fazer COMMIT
- fazer ROLLBACK

AULA 15 — Triggers

1. O que é uma Trigger?

Uma trigger é um bloco de código SQL que é executado automaticamente quando ocorre um evento numa tabela.

- Tu não chamas uma trigger
- O SGBD chama-a sozinho

Eventos típicos:

- `INSERT`
- `UPDATE`
- `DELETE`

2. Analogia simples

Trigger = alarme automático

- Evento: alguém abre a porta
- Trigger: alarme dispara

3. Quando uma trigger é executada?

Uma trigger está associada a:

- **Um evento**

- `INSERT`
- `UPDATE`
- `DELETE`

- **Um momento**

- `BEFORE` → antes da operação
- `AFTER` → depois da operação

Exemplo de leitura

“`AFTER INSERT ON aluno`”

- Depois de inserir um aluno, a trigger executa.

4. Tipos de triggers

- **BEFORE Trigger**

- executa antes da operação
- pode validar ou alterar dados

Exemplo:

- impedir notas negativas
- ajustar valores automaticamente

- **AFTER Trigger**

- executa **depois** da operação
- muito usada para:
 - logs
 - auditoria
 - atualizações em cascata

5. Exemplo simples de Trigger (conceitual)

```
CREATE TRIGGER verifica_nota
BEFORE INSERT ON aluno
FOR EACH ROW
BEGIN
    IF NEW.nota < 0 THEN
        SET NEW.nota = 0;
    END IF;
END;
```


Sempre que alguém tentar inserir uma nota negativa a trigger corrige automaticamente

6. NEW e OLD

Dentro de uma trigger tens acesso a:

- **NEW** → valores novos
- **OLD** → valores antigos

Exemplo

- **INSERT** → só **NEW**
- **DELETE** → só **OLD**
- **UPDATE** → ambos

7. Triggers e Integridade de Dados

Triggers são usadas para:

- garantir regras de negócio complexas
- manter consistência
- automatizar validações

Exemplo:

- atualizar saldo total
- manter histórico
- impedir alterações inválidas

8. Trigger vs CHECK vs Procedimento

CHECK	Trigger	Procedimento
Regra simples	Regra complexa	Ação explícita
Declarativa	Imperativa	Imperativa
Sem lógica	Com lógica	Com lógica
Executa sempre	Executa automaticamente	Executa quando chamada

- Usa CHECK sempre que possível
- Trigger só quando CHECK não chega

9. Riscos das Triggers ⚠

Triggers são poderosas, mas perigosas:

- Difíceis de depurar
- Executam “invisivelmente”
- Podem causar loops
- Podem afetar desempenho

10. Quando usar Triggers?

Usa triggers quando:

- a regra não pode ser expressa com CHECK
- precisa de reagir automaticamente a eventos
- a lógica pertence claramente à base de dados

AULA 16 — Interrogações Recursivas (WITH RECURSIVE)

1. O que é uma interrogação recursiva?

Uma **interrogação recursiva** é uma consulta que usa o resultado parcial da própria consulta para gerar novos resultados. Em termos simples:

- a consulta “chama-se a si própria”
- até não haver mais resultados

2. Quando precisamos de recursão em bases de dados?

Recursão é usada quando os dados têm estrutura hierárquica.

Exemplos clássicos

- Organigramas (chefe → subordinados)
- Pastas e subpastas
- Categorias e subcategorias
- Pré-requisitos de disciplinas
- Grafos simples

3. Exemplo típico de dados hierárquicos

Tabela funcionario

id	nome	chefe_id
1	Ana	NULL
2	Bruno	1
3	Carla	2

- `chefe_id` aponta para outro funcionário

- Isto cria uma auto-relação

4. Porque SQL normal não chega?

Pergunta: “Quem trabalha direta ou indiretamente para a Ana?”

- SELECT simples → só 1 nível
- JOIN → 1 nível
- JOIN JOIN JOIN... → impossível de generalizar

5. WITH RECURSIVE — a estrutura geral

Uma query recursiva tem **duas partes**:

1. **Caso base** (âncora)
2. **Caso recursivo**

- E junta tudo com `UNION ALL`.

Estrutura geral

```
WITH RECURSIVE nome_cte AS (  
    -- 1.Caso base  
    SELECT ...  
    FROM ...  
    WHERE ...  
  
    UNION ALL  
  
    -- 2. Caso recursivo  
    SELECT ...  
    FROM ...  
    JOIN nome_cte ON ...  
)  
SELECT * FROM nome_cte;
```

- nome_cte é uma CTE (Common Table Expression)

6. Caso base

Define onde a recursão começa

Exemplo: Começar pelo chefe principal (Ana)

```
SELECT id, nome, chefe_id  
FROM funcionario  
WHERE chefe_id IS NULL
```

Sem isto, a recursão não arranca.

7. Caso recursivo — expansão

Usa o resultado anterior para encontrar novos dados.

Exemplo: Encontrar funcionários cujo chefe está no resultado anterior

```
SELECT f.id, f.nome, f.chefe_id
FROM funcionario f
JOIN nome_cte n
ON f.chefe_id = n.id
```

Isto vai descendo na hierarquia.

8. Exemplo completo

```
WITH RECURSIVE hierarquia AS (
    -- Caso base
    SELECT id, nome, chefe_id
    FROM funcionario
    WHERE chefe_id IS NULL

    UNION ALL

    -- Caso recursivo
    SELECT f.id, f.nome, f.chefe_id
    FROM funcionario f
    JOIN hierarquia h
    ON f.chefe_id = h.id
)

SELECT * FROM hierarquia;
```

Resultado:

- Ana
- Bruno
- Carla

9. Como o SGBD executa isto?

1. Executa o **caso base**
2. Guarda o resultado
3. Executa o **caso recursivo**
4. Junta novos resultados
5. Repete até não haver mais linhas
6. Para automaticamente

10. UNION vs UNION ALL

- `UNION ALL` → mais rápido
- `UNION` → remove duplicados

Em recursão:

- usa-se **quase sempre** `UNION ALL`
- evitar custos desnecessários

AULA 17 — Indexação

1. O problema sem índices

Imagina uma tabela com 1 milhão de alunos:

```
SELECT *
FROM aluno
WHERE id = 123456;
```

Sem índice:

- o SGBD tem de ler linha a linha
- isto chama-se table scan
- é lento ❌

2. O que é um Índice?

Um índice é uma estrutura de dados auxiliar que permite encontrar linhas rapidamente. É como o índice de um livro:

- sem índice → folheias tudo
- com índice → vais direto à página

3. O que um índice guarda?

Normalmente um índice guarda:

- o valor da coluna indexada
- um ponteiro para a linha real da tabela

Exemplo: (`id = 123456`) → endereço da linha

4. Tipos principais de índices

- Índices ordenados (B+ Tree)

Características:

- dados ordenados
- permitem:
 - igualdade (`=`)
 - intervalos (`<`, `>`, `BETWEEN`)
 - ordenações (`ORDER BY`)

Muito usados em:

- chaves primárias
- chaves estrangeiras

- Índices Hash

Baseados em funções hash.

Características:

- muito rápidos para igualdade (`=`)
- não funcionam bem para intervalos

Exemplo: `WHERE id = 10`

Mau para: `WHERE id > 10`

5. Índice vs Chave Primária

Chave Primária	Índice
Conceito lógico	Estrutura física
Garante unicidade	Acelera acesso
Regra de integridade	Otimização
Sempre única	Pode não ser

- Toda chave primária cria um índice,
- mas nem todo índice é chave primária.

6. Criar um índice em SQL

```
CREATE INDEX idx_aluno_nome  
ON aluno(nome);
```

Agora consultas por `nome` ficam mais rápidas.

Índice único

```
CREATE UNIQUE INDEX idx_email  
ON aluno(email);
```

7. Quando um índice é usado?

Um índice costuma ser usado quando:

- aparece no **WHERE**
- aparece no **JOIN**
- aparece no **ORDER BY**
- aparece no **GROUP BY**

8. Quando um índice NÃO ajuda?

Colunas com poucos valores distintos (ex.: sexo = M/F)

-Tabelas muito pequenas

-Colunas usadas raramente

-Uso de funções na coluna

```
WHERE UPPER(nome) = 'ANA';
```

9. Índices têm custo

Índices não são gratuitos.

Custos:

- ocupam espaço
- tornam **INSERT**, **UPDATE**, **DELETE** mais lentos
- precisam de manutenção

10. Índices e Otimização

O otimizador escolhe usar índice ou não depende de:

- estatísticas
- seletividade
- custo estimado

11. Índices e EXPLAIN

```
EXPLAIN SELECT * FROM aluno WHERE id = 10;
```

Se vires:

- **Index Scan** → índice usado
- **Seq Scan** → varrimento completo

AULA 18 — Transações e Concorrência

1. O que é uma Transação?

Uma transação é uma sequência de operações que deve ser executada como uma única unidade lógica.

2. Exemplo clássico (banco)

Transferência de dinheiro:

1. retirar 100€ da conta A
2. adicionar 100€ à conta B

Isto não pode ficar a meio.

Sem transações:

- sistema falha após passo 1
- dinheiro desaparece

3. Estados de uma Transação

Uma transação pode estar em vários estados:

- Ativa → a executar
- Parcialmente confirmada
- Confirmada (COMMIT)
- Abortada (ROLLBACK)

4. Propriedades ACID

As transações têm de cumprir **ACID**:

- Atomicidade

Tudo ou nada

- Se algo falhar → tudo é desfeito

- Consistência

A base de dados passa de um estado válido para outro válido

- regras continuam válidas
- restrições respeitadas

- Isolamento

Transações não interferem umas com as outras

- cada transação “acha” que está sozinha

- Durabilidade

Depois de confirmado, não se perde

- mesmo com falhas de energia

5. SQL e Transações

Comandos principais:

```
BEGIN;          COMMIT;      ROLLBACK;
```

Exemplo

```
BEGIN;  
UPDATE conta SET saldo = saldo - 100 WHERE id = 1;  
UPDATE conta SET saldo = saldo + 100 WHERE id = 2;  
COMMIT;
```

Se algo falhar antes do COMMIT → ROLLBACK automático.

6. O que é Concorrência?

Concorrência acontece quando várias transações executam ao mesmo tempo

Isto é normal em sistemas reais:

- múltiplos utilizadores
- múltiplas aplicações
- múltiplos servidores

7. Problemas clássicos de concorrência ⚠

Sem controlo, surgem erros graves:

- Atualização perdida (Lost Update)

- T1 lê valor
- T2 lê o mesmo valor
- ambas atualizam
- uma atualização perde-se

- Leitura suja (Dirty Read)

- T1 altera valor
- T2 lê valor ainda não confirmado

- T1 faz rollback
- T2 leu lixo

- Leitura não repetível

- T1 lê um valor
- T2 altera e confirma
- T1 lê novamente → valor diferente

- Leitura fantasma

- T1 executa consulta
- T2 insere novas linhas
- T1 repete consulta → aparecem linhas novas

8. Porque estes problemas são graves?

- resultados inconsistentes
- decisões erradas
- dados incorretos

Especialmente perigosos em:

- bancos
- reservas
- faturação

9. Isolamento e níveis de isolamento

Para lidar com concorrência, o SGBD define níveis de isolamento.

Mais isolamento = menos problemas

Mais isolamento = menos desempenho

10. Ligação com aulas anteriores

- Procedimentos → usam transações
- Triggers → executam dentro de transações
- Índices → afetam concorrência
- Otimização → considera locks

AULA 19 — Controlo de Concorrência e Níveis de Isolamento

1. Porque precisamos de controlo de concorrência?

Controlo de concorrência é o conjunto de mecanismos que o SGBD usa para garantir que o resultado final é correto, como se as transações tivessem sido executadas uma de cada vez.

2. Escalonamento (Scheduling)

Escalonamento = ordem em que as operações de várias transações são intercaladas.

Exemplo

```
T1: READ(A)
T2: READ(A)
T1: WRITE(A)
T2: WRITE(A)
```

Isto é um escalonamento **concorrente**.

3. Escalonamento em série

Escalonamento em série:

- uma transação executa do início ao fim
- depois começa a outra

Exemplo: T1 → T2

- Sempre correto
- Pouco eficiente

4. Serializabilidade

Um escalonamento é **serializável** se produz o mesmo resultado que algum escalonamento em série. Mesmo que as operações estejam intercaladas.

Tipos importantes

- Serializabilidade por conflitos (a mais importante)
- Serializabilidade por visão (menos cobrada)

5. Conflitos entre operações

Duas operações **entram em conflito** se:

- são de transações diferentes
- acedem ao mesmo dado
- pelo menos uma é escrita

Exemplos de conflito:

- READ / WRITE
- WRITE / READ
- WRITE / WRITE

6. Como o SGBD garante serializabilidade?

Através de **mecanismos de controlo**, como:

- Bloqueios (locks)
- Protocolos de bloqueio
- Versionamento (MVCC)

7. Níveis de Isolamento (SQL Standard)

O SQL define **4 níveis de isolamento**.

- READ UNCOMMITTED

Permite:

- leituras sujas ✗
- leituras não repetíveis ✗
- fantasmas ✗

Quase nunca usado.

- READ COMMITTED

Evita:

- leituras sujas ✓

Permite:

- leituras não repetíveis ✗
- fantasmas ✗

Muito comum (ex.: PostgreSQL default).

- REPEATABLE READ

Evita:

- leituras sujas ✓
- leituras não repetíveis ✓

Permite:

- fantasmas ✗ (dependendo do SGBD)

Mais isolamento, menos concorrência.

- SERIALIZABLE (mais forte)

Evita:

- todos os problemas ✓

Comporta-se como se as transações fossem executadas em série.

-Menor desempenho

-Máxima segurança

8. Tabela-resumo

Nível	Sujas	Não repetíveis	Fantasma
READ UNCOMMITTED	✗	✗	✗
READ COMMITTED	✓	✗	✗
REPEATABLE READ	✓	✓	✗
SERIALIZABLE	✓	✓	✓

9. Definir nível de isolamento em SQL

Exemplo: `SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;`

Ou: `SET TRANSACTION ISOLATION LEVEL READ COMMITTED;`

10. Isolamento vs desempenho

Regra prática:

- Mais isolamento → menos erros, menos concorrência
- Menos isolamento → mais desempenho, mais risco

Escolha depende da aplicação:

- banco → SERIALIZABLE
- relatórios → READ COMMITTED

11. Ligação com bloqueios (prévia)

Por trás dos níveis:

- o SGBD usa **locks**
- ou **versionamento**

AULA 20 — Bloqueios (Locks) e Protocolo de Duas Fases (2PL)

1. O que é um Bloqueio (Lock)?

Um bloqueio é um mecanismo que controla o acesso concorrente a um dado. Antes de:

- ler
- escrever

2. Tipos principais de bloqueios

- Bloqueio Partilhado (Shared Lock — S)

Usado para **leitura**.

- várias transações podem ler ao mesmo tempo
- ninguém pode escrever

Exemplo:

`T1: READ(A) → S(A)`

`T2: READ(A) → S(A)`

- Bloqueio Exclusivo (Exclusive Lock — X)

Usado para **escrita**.

- só uma transação pode ter o lock
- ninguém mais pode ler nem escrever

Exemplo:

T1: WRITE(A) → X(A)

3. Tabela de compatibilidade de locks

	S	X
S	✓	✗
X	✗	✗

Só S com S é compatível.

4. Como funciona o controlo com locks?

Fluxo típico:

1. transação pede lock
2. se for compatível → concede
3. se não for → espera

O SGBD gere filas de espera.

5. O Protocolo de Duas Fases (2PL)

2PL garante serializabilidade por conflitos.

Divide a execução de uma transação em duas fases:

- Fase de Crescimento (Growing Phase)

- a transação só adquire locks
- não pode libertar nenhum

- Fase de Redução (Shrinking Phase)

- a transação só liberta locks
- não pode adquirir novos

Uma vez libertado um lock, não pode pedir mais nenhum.

6. Porque o 2PL é importante?

Garante que:

- o escalonamento é **serializável**
- não há conflitos perigosos

Sem 2PL:

- resultados incorretos
- inconsistência

7. 2PL Estrito (Strict 2PL)

Regras:

- locks exclusivos (X) só são libertados no COMMIT ou ROLLBACK

Evita:

- leituras sujas
- cascata de rollbacks

8. Deadlock (bloqueio circular)

Deadlock ocorre quando duas ou mais transações ficam à espera umas das outras indefinidamente.

Exemplo clássico

T1 bloqueia A, quer B

T2 bloqueia B, quer A

9. Como o SGBD lida com deadlocks?

Existem duas abordagens:

- Detecção

- o SGBD constrói um grafo de espera
- se houver ciclo → deadlock
- aborta uma transação

- Prevenção

- regras de prioridade
- ordenação de locks
- abortar cedo

10. Locks vs Níveis de Isolamento

Os níveis de isolamento:

- são uma abstração
- por baixo usam:
 - locks
 - ou versionamento (MVCC)

AULA 21 — Bases de Dados Distribuídas

1. O que é uma Base de Dados Distribuída?

Uma base de dados distribuída é uma base de dados cujos dados estão armazenados em vários computadores (nós) ligados por uma rede.

Para o utilizador:

- parece uma única base de dados

Internamente:

- os dados estão espalhados

2. Porque usar Bases de Dados Distribuídas?

As bases de dados “normais” (centralizadas) têm limites:

- não escalam bem
- ponto único de falha
- latência para utilizadores distantes

Bases de dados distribuídas resolvem isto:

- **Escalabilidade** (mais nós → mais capacidade)
- **Disponibilidade** (se um nó falhar, outros continuam)
- **Desempenho** (dados mais perto do utilizador)
- **Tolerância a falhas**

3. O que é um Nó?

Nó = um computador/servidor que:

- armazena parte dos dados
- executa parte do SGBD

Uma base distribuída = **conjunto de nós**.

4. Transparência

Idealmente, o utilizador não deve perceber que a BD é distribuída.

Tipos de transparência:

- de localização (não sabes onde estão os dados)
- de fragmentação
- de replicação
- de falhas

Tudo isto é responsabilidade do SGBD distribuído.

5. Arquiteturas de Bases de Dados Distribuídas

- Shared-Nothing (mais comum)

Cada nó tem:

- CPU própria
- memória própria
- disco próprio

Não partilham recursos físicos.

- Escala muito bem
- Muito usada em sistemas modernos

- Shared-Disk

Vários nós:

- partilham o mesmo disco
- têm CPUs e memória próprias

- Escala pior
- Mais complexa

- Shared-Memory

Tudo partilhado:

- memória
- disco

6. Fragmentação (dividir dados)

Fragmentação = dividir uma tabela em partes.

- Fragmentação Horizontal

- divide **linhas**

Exemplo:

- alunos de 2024 num nó
- alunos de 2025 noutro

- Fragmentação Vertical

- divide **colunas**

Exemplo:

- dados pessoais num nó
- dados académicos noutro

- Fragmentação Mista

- combinação das duas

7. Replicação (copiar dados)

Replicação = manter cópias dos dados em vários nós.

Vantagens

- alta disponibilidade
- tolerância a falhas
- leitura mais rápida

Desvantagens

- custo de sincronização
- problemas de consistência

8. Fragmentação vs Replicação

Fragmentação	Replicação
Divide dados	Duplica dados
Menos redundância	Mais redundância
Escalabilidade	Disponibilidade
Cada dado num sítio	Dados em vários sítios

9. Desafios das Bases de Dados Distribuídas

- Latência de rede
- Falhas parciais
- Sincronização
- Transações distribuídas
- Consistência

AULA 22 — Transações Distribuídas e Two-Phase Commit (2PC)

1. O que é uma Transação Distribuída?

Transação distribuída é uma transação que envolve **várias bases de dados / nós diferentes**

Cada nó:

- executa uma parte da transação
- tem o seu próprio SGBD

2. Porque transações distribuídas são difíceis?

Num sistema centralizado:

- um único SGBD
- um único log
- decisão simples

Num sistema distribuído:

- falhas de rede
- nós podem cair
- mensagens podem atrasar
- decisões têm de ser coordenadas

3. Exemplo clássico

Banco com duas agências:

- Conta A → Nó 1
- Conta B → Nó 2

Transferência:

1. debitar A (Nó 1)
2. creditar B (Nó 2)

Ambos **têm de concordar** no resultado final.

4. A solução clássica: Two-Phase Commit (2PC)

2PC é um protocolo que garante **atomicidade** em transações distribuídas

Ou seja:

- ou todos confirmam (COMMIT)

- ou todos abortam (ROLLBACK)

5. Papéis no 2PC (muito importante ⚠)

Existem dois tipos de participantes:

-Coordenador

- controla o protocolo
- toma a decisão final
- comunica com os participantes

-Participantes

- executam a sua parte da transação
- respondem ao coordenador

6. Fase 1 — Prepare (ou Voting Phase)

O coordenador pergunta: “Consegues confirmar a transação?”

Passos:

1. coordenador envia **PREPARE**
2. cada participante:
 - executa localmente
 - guarda tudo no log
 - responde:
 - **YES** (pode confirmar)
 - **NO** (tem de abortar)

7. Fase 2 — Commit (ou Decision Phase)

Caso todos respondam YES:

- coordenador envia **COMMIT**
- todos confirmam localmente

Se alguém responder NO:

- coordenador envia **ABORT**
- todos fazem rollback

8. Diagrama mental do 2PC

PREPARE

Coordenador → Participantes
← YES / NO

COMMIT / ABORT

Coordenador → Participantes

9. O que o 2PC garante?

Atomicidade distribuída

Consistência global

Mesmo com múltiplos nós.

10. Limitações do 2PC

Bloqueante

- se o coordenador falhar após PREPARE
- participantes ficam à espera

-Latência elevada

- Muitos logs e mensagens

- Escala mal em sistemas grandes

11. Falhas possíveis no 2PC

- Falha de um participante

- coordenador decide ABORT

- Falha do coordenador (pior caso)

- participantes ficam bloqueados
- não sabem se devem COMMIT ou ABORT

12. Alternativas ao 2PC (contexto)

Para sistemas grandes:

- 3PC (Three-Phase Commit)
- Protocolos BASE
- Consistência eventual
- Sistemas NoSQL

AULA 23 — NoSQL e o Teorema CAP

1. Porque surgiu o NoSQL?

As bases de dados relacionais são ótimas, mas em sistemas muito grandes surgem limites:

- escalam mal horizontalmente
- transações distribuídas são caras
- esquema rígido
- latência elevada à escala global

Empresas como Google, Amazon, Facebook precisavam de:

- escalar para milhões/biliões de dados
- aceitar falhas
- responder rápido

2. O que é NoSQL?

NoSQL é uma família de bases de dados que:

- não seguem estritamente o modelo relacional
- não usam SQL clássico
- priorizam **escalabilidade, disponibilidade e desempenho**

3. Principais diferenças: SQL vs NoSQL

SQL (Relacional)	NoSQL
Esquema fixo	Esquema flexível
ACID	BASE
JOINS	Sem JOINS (ou limitados)
Forte consistência	Consistência eventual
Escala vertical	Escala horizontal
Estrutura rígida	Estrutura flexível

4. Tipos principais de NoSQL

- Chave-Valor

- estrutura simples: (chave → valor)
- muito rápido

Exemplo:

```
"user123" → { nome: "Ana", idade: 20 }
```

Usado em:

- caches

- sessões

- Documentos

- dados em JSON / BSON
- estrutura flexível

Exemplo:

```
{
  "nome": "Ana",
  "curso": "EI",
  "disciplinas": ["BD", "SO"]
}
```

- Colunas (Wide-Column)

- inspirado no BigTable
- otimizado para grandes volumes

Usado em:

- analytics
- Big Data

- Grafos

- nós e arestas
- relações complexas

Usado em:

- redes sociais
- recomendações
- caminhos

5. O Teorema CAP

O **Teorema CAP** diz que num sistema distribuído **não é possível garantir ao mesmo tempo:**

- **C — Consistência**
- **A — Disponibilidade**
- **P — Tolerância a Partições**

Só pode-se escolher **2 de 3**.

6. O que significa cada letra?

- Consistência (C)

- todos os nós veem os mesmos dados
- leitura devolve sempre o valor mais recente

- Disponibilidade (A)

- o sistema responde sempre
- mesmo com falhas

- Tolerância a Partições (P)

- sistema continua a funcionar mesmo com falhas de rede

- Em sistemas distribuídos reais, **P é obrigatório**.

7. As três combinações possíveis

- CP — Consistência + Partição

- pode negar pedidos para manter consistência
- usado quando correção é crítica

- AP — Disponibilidade + Partição

- sistema responde sempre
- dados podem estar temporariamente inconsistentes

Consistência eventual

- CA — Consistência + Disponibilidade

- só funciona sem partições, **não é realista** em sistemas distribuídos

8. ACID vs BASE

- ACID (SQL)

- Atomicidade
- Consistência
- Isolamento
- Durabilidade

Forte, seguro, caro

- BASE (NoSQL)

- **Basically Available**
- **Soft state**
- **Eventually consistent**

Rápido, escalável, flexível

9. Consistência Eventual

Significa que se não houver novas atualizações, todos os nós acabam por ficar consistentes

Exemplo:

- post aparece agora
- likes atualizam aos poucos

Aceitável em:

- redes sociais
- feeds
- recomendações

Não aceitável em:

- bancos
- pagamentos

10. Quando usar SQL vs NoSQL?

Usa SQL quando:

- dados bem estruturados
- integridade forte
- transações críticas
- consistência imediata

Usa NoSQL quando:

- grande volume de dados
- esquema flexível
- alta disponibilidade
- latência baixa
- consistência eventual aceitável

AULA 24 — Data Warehousing e Business Intelligence (BI)

1. Problema inicial: analisar dados operacionais

As bases de dados normais (OLTP) são ótimas para:

- inserir
- atualizar
- apagar
- transações rápidas

Exemplos

- registrar compras
- atualizar notas

- transferências bancárias

Mas são más para:

- relatórios grandes
- estatísticas históricas
- análises complexas

2. OLTP vs OLAP

- OLTP — Online Transaction Processing

- dados atuais
- muitas escritas
- consultas simples
- foco: operação

Ex.: sistema de vendas

- OLAP — Online Analytical Processing

- dados históricos
- poucas escritas
- consultas pesadas
- foco: análise

Ex.: relatórios de vendas anuais

3. O que é um Data Warehouse?

Um Data Warehouse é uma base de dados orientada à análise, com estas características clássicas:

- **Orientado por assunto** (vendas, clientes, tempo)
 - **Integrado** (dados de várias fontes)
 - **Histórico** (dados ao longo do tempo)
 - **Não volátil** (não se apaga nem atualiza)
- Dados entram, **não são alterados**.

4. De onde vêm os dados? (ETL)

Os dados não nascem no Data Warehouse.

Processo clássico: **ETL**

- Extract

- extrair dados de sistemas operacionais
- ERP, CRM, bases SQL, ficheiros, APIs

- Transform

- limpar dados
- corrigir erros
- normalizar formatos
- aplicar regras de negócio

- Load

- carregar dados no Data Warehouse

5. Modelação em Data Warehousing (diferente!)

Aqui não usamos o modelo relacional clássico normalizado. Usamos modelos dimensionais.

- Tabela de Factos

- métricas numéricas
- eventos

Exemplo:

- vendas
- quantidades
- valores

- Tabelas de Dimensão

- contexto
- descrição

Exemplos:

- tempo
- produto
- cliente
- loja

6. Esquema em Estrela

O mais comum.

- tabela de factos no centro
- dimensões à volta

Consultas mais simples e rápidas.

7. Esquema em Floco de Neve

- dimensões normalizadas
- mais tabelas
- menos redundância

Consultas mais complexas

8. O que é um Data Mart?

Data Mart é: um subconjunto do Data Warehouse focado num departamento

Exemplos:

- vendas
- marketing
- finanças

Menor, mais específico, mais rápido de implementar.

9. Business Intelligence (BI)

BI é o conjunto de ferramentas e técnicas para:

- explorar dados
- criar relatórios
- dashboards
- apoiar decisões

Exemplos de outputs BI

- gráficos
- KPIs
- tabelas dinâmicas
- painéis interativos

O utilizador não escreve SQL complexo.

10. SQL analítico (diferença importante)

Consultas típicas de BI:

- grandes JOINS
- GROUP BY pesados
- agregações
- períodos longos

Por isso:

- Data Warehouses são otimizados para leitura
- usam índices diferentes
- às vezes usam colunas (columnar storage)

11. Porque separar operacional de analítico?

Misturar tudo causa problemas:

- relatórios lentos
- impacto em transações
- concorrência pesada

Separar:

- melhor desempenho
- menos risco
- arquitetura limpa